

Slip 3

Q.1) Print the type of file and inode number where file name accepted through Command Line

[10 Marks]

```
#include <stdio.h>    // For standard I/O functions like printf, fprintf
#include <sys/stat.h> // For the stat() function and struct stat
#include <sys/types.h> // For basic system data types
#include <unistd.h>    // For basic system data types (though often included
via <sys/types.h>)
#include <stdlib.h>    // For exit()
void print_file_type(mode_t mode) {
    printf("File Type: ");
    if (S_ISREG(mode)) {
        printf("Regular File\n");
    } else if (S_ISDIR(mode)) {
        printf("Directory\n");
    } else if (S_ISLNK(mode)) {
        printf("Symbolic Link\n");
    } else if (S_ISCHR(mode)) {
        printf("Character Device\n");
    } else if (S_ISBLK(mode)) {
        printf("Block Device\n");
    } else if (S_ISFIFO(mode)) {
        printf("FIFO (Named Pipe)\n");
    } else if (S_ISSOCK(mode)) {
        printf("Socket\n");
    } else {
        printf("Unknown Type\n");
    }
}
int main(int argc, char *argv[]) {
    if (argc != 2) {
        fprintf(stderr, "Usage: %s <filename>\n", argv[0]);
        exit(EXIT_FAILURE);
    }
    const char *filename = argv[1];
    struct stat file_stat;
specifically needed.
    if (stat(filename, &file_stat) == -1) {
        // stat() returns -1 on failure
        perror("stat failed");
        exit(EXIT_FAILURE);
    }
}
```

```

printf("--- File Information for: %s ---\n", filename);
printf("Inode Number: %ld\n", (long)file_stat.st_ino);
print_file_type(file_stat.st_mode);
printf("-----\n");
return 0;
}

```

Q.2) Write a C program which creates a child process to run linux/ unix command or any user defined program. The parent process set the signal handler for death of child signal and Alarm signal. If a child process does not complete its execution in 5 second then parent process kills child process.[20 Marks]

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

pid_t cpid;

void sig_handler(int sig) {
    wait(NULL);
    printf("Child is terminated\n");
    exit(0);
}

void sig_alarm(int sig) {
    printf("Child took too long, terminating...\n");
    kill(cpid, SIGKILL);
    exit(0);
}

int main() {
    // Set up signal handlers
    signal(SIGALRM, sig_alarm); // Correct signal for alarm
    signal(SIGCHLD, sig_handler); // Handle child termination

    cpid = fork();

    if (cpid < 0) {
        perror("fork failed");
        exit(EXIT_FAILURE);
    }

    if (cpid == 0) {
        // Child process

```

```
        execlp("sleep", "sleep", "10", NULL);
        perror("execlp failed");
        exit(EXIT_FAILURE);
    } else {
        // Parent process
        alarm(5); // Set timeout
        pause();  // Wait for signal
    }

    return 0;
}
```